

INFORME TÉCNICO · PATRONES DE DISEÑO

Patrón Singleton: instancia única del Banco Central

Integración del patrón Singleton en la arquitectura de Quantum Core

Implementación y modelado UML de una instancia única de control para la configuración y los datos globales de la Fintech, y análisis de los riesgos de prescindir del patrón en componentes críticos

ESTUDIANTE	Deibis Zuluaga Baena
NÚCLEO	Fundamentos de Software · FUNSO_2_N_H
ACTIVIDAD	El patrón de diseño Singleton
HERRAMIENTA	Draw.io Integration · Visual Studio Code
ENTREGABLES	Diagrama de clases (PNG, SVG y .drawio), código fuente e informe
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
FECHA DE ENTREGA	27 de septiembre de 2026

Objetivo del trabajo

Objetivo: aplicar el patrón de diseño Singleton en el módulo central de Quantum Core, de modo que exista una única instancia de control (Banco Central) que administre la configuración y los datos globales del sistema, asegure su integridad y optimice el uso de memoria.

CÓDIGO ORIGINAL DE LA ACTIVIDAD

El archivo `banco_central.py` se publicó en el enunciado de Canvas y en el repositorio del curso, con algunas diferencias entre ambas versiones. El modelo integra los elementos de las dos: el atributo privado `__instancia` y los fondos totales de la versión de Canvas, y las reservas, el método `emitir_moneda()` y la demostración con `id()` del repositorio. Para enriquecer el ejercicio se adicionó una configuración global, identificada en el código con el comentario `[AGREGADO]`.

RELACIÓN CON EL MODELO DE LA QUANTUM WALLET

El diagrama de clases desarrollado previamente para la Quantum Wallet se conserva completo y se amplía con la clase `BancoCentral`. La `wallet` consulta al Banco Central la moneda del sistema y el límite por transacción, lo que integra el Singleton a la arquitectura existente en lugar de presentarlo como un componente aislado.

CRITERIO PARA LOS TIPOS DE DATOS

Python no exige declarar tipos. En el diagrama se registraron según el uso de cada dato en el código: montos, fondos y reservas como `float`, textos como `str` y la configuración como `dict`.

Draw.io Integration en Visual Studio Code

El diagrama se elaboró con la extensión **Draw.io Integration**, desarrollada por Henning Dieterichs, que incorpora el editor de diagrams.net (draw.io) en Visual Studio Code. De esta manera, el modelado se realiza en el mismo entorno de desarrollo en el que se encuentra el código analizado. Se utilizó la versión 1.11.0 de la extensión en Visual Studio Code sobre macOS; el diagrama se almacena en formato `.drawio` y se exporta a PNG y SVG.

Funcionamiento

ETAPA	DESCRIPCIÓN
Instalación	Se instala desde el panel de Extensiones de Visual Studio Code. No requiere cuenta de usuario ni software adicional.
Apertura	Al abrir un archivo <code>.drawio</code> desde el explorador del proyecto, Visual Studio Code presenta el editor gráfico en lugar del contenido del archivo.
Modelado	El panel lateral contiene las bibliotecas de formas. La biblioteca UML incluye la caja de clase de tres secciones y los conectores de herencia, composición y agregación.
Relaciones	Los conectores permanecen vinculados a las clases: al reubicar una clase, las relaciones se ajustan de forma automática. El tipo de terminación se define en el panel de estilo.
Exportación	La opción <i>File</i> → <i>Export</i> genera la imagen en formato PNG o SVG. Las figuras de este informe se obtuvieron de esa exportación.
Versionado	El archivo <code>.drawio</code> se almacena en formato XML, por lo que Git registra sus cambios del mismo modo que los del código fuente.

CRITERIO DE SELECCIÓN

La herramienta mantiene el diagrama en el mismo repositorio y en el mismo editor que el código, sin depender de una plataforma en línea ni de una cuenta externa. Además, el archivo fuente permanece editable, lo que permite incorporar nuevas clases al modelo sin necesidad de rehacerlo.

El patrón Singleton

Un patrón de diseño es una solución reutilizable a un problema recurrente en el diseño de software; no es un fragmento de código para copiar, sino una estructura de solución. El patrón Singleton garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

Problema en Quantum Core

El Banco Central coordina los fondos y la configuración de la Fintech. Si cada operación creara un Banco Central nuevo, cada objeto tendría sus propios fondos y reservas: los saldos dejarían de ser consistentes y la memoria se ocuparía con copias innecesarias. El sistema requiere que exista una sola instancia, compartida por todos los componentes.

Mecanismo de implementación en Python

MÉTODO

FUNCIÓN EN LA CREACIÓN DEL OBJETO

<code>__new__</code>	Crea el objeto y lo devuelve. Se ejecuta primero, por lo que es el punto donde se decide si se crea una instancia nueva o se entrega la existente.
<code>__init__</code>	Inicializa un objeto que ya fue creado. Se ejecuta cada vez que se invoca la clase, incluso cuando <code>__new__</code> devuelve la instancia existente; por esa razón la implementación no lo define.

```
class BancoCentral:
    __instancia = None                # atributo de clase (estático)

    def __new__(cls):
        if cls.__instancia is None:  # solo la primera vez
            cls.__instancia = super(BancoCentral, cls).__new__(cls)
            cls.__instancia.fondos_totales = 1000000
            cls.__instancia.reservas = 0
        return cls.__instancia        # siempre el mismo objeto
```

RESULTADO

La primera invocación de `BancoCentral()` crea el objeto e inicializa sus datos; las siguientes devuelven exactamente el mismo objeto, con la misma dirección de memoria. La inicialización se realiza dentro de `__new__` y no en `__init__`, lo que evita que los fondos y las reservas se reinicien con cada invocación.

Arquitectura con el Banco Central

La Figura 1, presentada en la página siguiente, muestra el modelo completo: la Quantum Wallet con sus cinco clases y la nueva clase `BancoCentral`, que aplica el patrón Singleton y se relaciona con la `Wallet` mediante una dependencia.

Representación del patrón Singleton

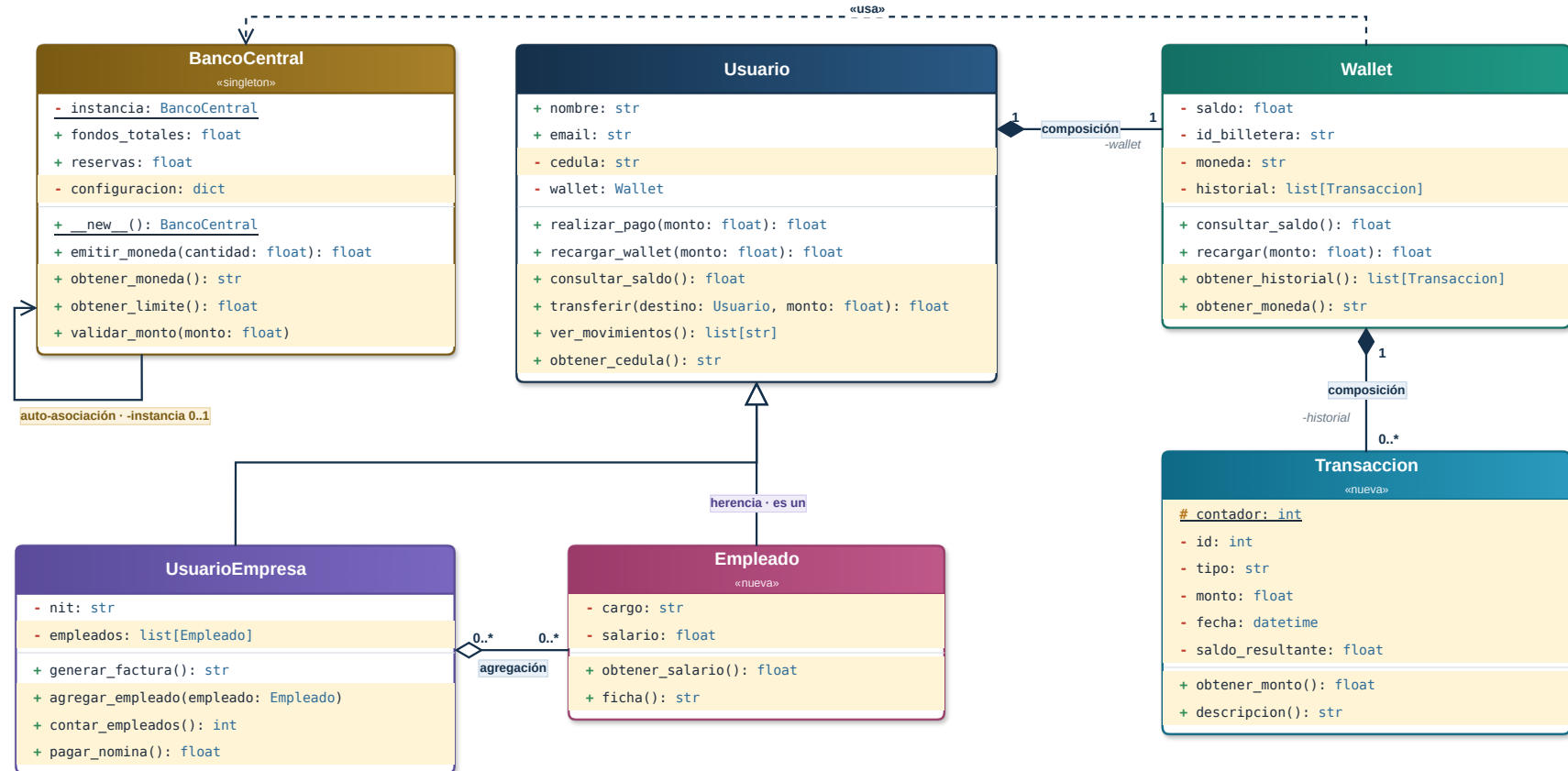
ELEMENTO	REPRESENTACIÓN EN EL DIAGRAMA
Estereotipo «singleton»	Debajo del nombre de la clase, indica que <code>BancoCentral</code> admite una sola instancia.
Atributo estático	<code>- instancia: BancoCentral</code> aparece subrayado porque pertenece a la clase y no a cada objeto. El signo <code>-</code> indica que es privado (<code>__instancia</code>).
Punto de creación	<code>+ __new__(): BancoCentral</code> aparece subrayado porque se ejecuta sobre la clase antes de que exista el objeto.
Auto-asociación	Línea que sale de <code>BancoCentral</code> y regresa a la misma clase, con multiplicidad <code>0..1</code> : la clase guarda una referencia a su única instancia, que no existe hasta la primera invocación.
Dependencia «usa»	Línea punteada desde <code>Wallet</code> hacia <code>BancoCentral</code> : la billetera consulta la moneda y el límite por transacción sin almacenar una referencia al banco.

Convención de colores

ELEMENTO DEL DIAGRAMA	SIGNIFICADO
Filas con fondo blanco	Atributos y métodos del código original entregado en las actividades : las clases de la Quantum Wallet y el <code>BancoCentral</code> tal como fueron planteados.
Filas con fondo amarillo	Atributos y métodos adicionados para enriquecer el ejercicio , entre ellos la configuración global del Banco Central.
Clases con la etiqueta «nueva»	<code>Transaccion</code> y <code>Empleado</code> : clases adicionadas completas en el modelo de la Quantum Wallet.

Quantum Wallet · Diagrama de clases UML con patrón Singleton

Arquitectura lógica del módulo de usuarios y del Banco Central de Quantum Core



CÓMO LEER EL DIAGRAMA

- + **Público**: sin guion bajo (self.nombre)
- **Privado**: doble guion bajo (self.__saldo)
- # **Protegido**: un guion bajo (_contador)
- abc **Estático**: subrayado, es de la clase

- ▷ **Herencia**: triángulo hueco hacia la clase padre
- **Dependencia**: línea punteada, una clase usa a otra
- ↪ **Auto-asociación**: la clase guarda su propia instancia
- ◆ **Composición**: la parte no existe sin su dueño
- ◇ **Agregación**: la parte existe por su cuenta
- 0..* **Multiplicidad**: cero o muchos; 1 = exactamente uno

- Fondo amarillo: adicionado para enriquecer el ejercicio
- Fondo blanco: código original de la actividad
- «nueva» clase adicionada completa
- «singleton» clase de instancia única
- UsuarioEmpresa y Empleado heredan de Usuario todos sus atributos y métodos.

Figura 1. Diagrama de clases de la Quantum Wallet con el patrón Singleton. Fondo blanco: código original de la actividad; fondo amarillo: elementos adicionados para enriquecer el ejercicio. Elaboración propia en Draw.io Integration para Visual Studio Code.

La clase BancoCentral en Quantum Core

La tabla presenta los miembros de la clase, su origen y su función dentro del sistema.

MIEMBRO	ORIGEN	FUNCIÓN
- <code>instancia</code> (estático)	Canvas	Guarda la única instancia del banco. Es privado para que ninguna otra clase pueda reemplazarla.
+ <code>fondos_totales</code>	Canvas	Fondos del Banco Central, inicializados una sola vez en 1.000.000.
+ <code>reservas</code>	Repositorio	Moneda emitida por el banco; parte en cero.
- <code>configuracion</code>	Adicionado	Diccionario con la moneda del sistema y el límite por transacción.
+ <code>__new__()</code>	Canvas y repositorio	Crea la instancia la primera vez y la devuelve en todas las invocaciones.
+ <code>emitir_moneda(cantidad)</code>	Repositorio	Suma la cantidad a las reservas y devuelve el total.
+ <code>obtener_moneda()</code>	Adicionado	Devuelve la moneda oficial del sistema.
+ <code>obtener_limite()</code>	Adicionado	Devuelve el monto máximo permitido por transacción.
+ <code>validar_monto(monto)</code>	Adicionado	Genera <code>ValueError</code> si el monto supera el límite.

Decisiones de diseño

DECISIÓN	JUSTIFICACIÓN
Instancia privada (<code>__instancia</code>)	El repositorio usa <code>__instancia</code> (protegido) y Canvas <code>__instancia</code> (privado). Se adoptó la versión privada porque impide que una clase externa reemplace o anule la instancia única, que es la garantía central del patrón.
Sin método <code>__init__</code>	Si existiera, se ejecutaría en cada invocación y reiniciaría los fondos y las reservas. Toda la inicialización ocurre una sola vez dentro de <code>__new__</code> .
Integración con <code>wallet</code>	La billetera invoca <code>BancoCentral()</code> para obtener la moneda y validar el límite en <code>recargar()</code> . Como todas las billeteras reciben la misma instancia, todas aplican la misma configuración.

Encapsulamiento, responsabilidad única y bajo acoplamiento

La implementación se evaluó frente a tres principios de diseño de software.

PRINCIPIO

APLICACIÓN EN LA IMPLEMENTACIÓN

Encapsulamiento

La instancia única y la configuración son privadas. Las demás clases solo acceden a ellas por métodos públicos (`obtener_moneda()` , `obtener_limite()` , `validar_monto()`), por lo que ninguna puede alterar la configuración global ni reemplazar el banco.

Responsabilidad única

`BancoCentral` concentra lo que es global al sistema: fondos, emisión de moneda y configuración. El manejo del saldo de cada usuario permanece en `wallet` , y la `wallet` no define reglas globales: las consulta.

Bajo acoplamiento

La relación entre `wallet` y `BancoCentral` es una dependencia, no una asociación: la billetera no guarda una referencia al banco ni conoce su estructura interna, solo sus métodos públicos. Un cambio en la forma de almacenar la configuración no afecta a la billetera.

Limitaciones del patrón

El Singleton es conocido también como un anti-patrón cuando se usa en exceso, porque introduce estado global en el sistema. En esta implementación se identifican tres consideraciones:

LIMITACIÓN

TRATAMIENTO

Estado global

Cualquier parte del sistema puede acceder al banco. Se mitiga exponiendo solo métodos de consulta y validación, sin métodos que modifiquen la configuración.

Pruebas

La instancia persiste durante toda la ejecución, por lo que las pruebas comparten estado. Una mejora es incorporar un método de reinicio exclusivo para el entorno de pruebas.

Concurrencia

Con varios hilos, dos invocaciones simultáneas podrían crear dos instancias. En un entorno concurrente la creación debe protegerse con un bloqueo (`threading.Lock`).

Gestión centralizada de la configuración

El Banco Central actúa como gestor de configuración de Quantum Core. Los datos que deben ser iguales para todo el sistema se almacenan una sola vez en su instancia única.

DATO GLOBAL	VALOR	USO EN EL SISTEMA
Moneda	"COP"	Cada <code>wallet</code> la obtiene al crearse, de modo que todas las billeteras operan en la misma moneda.
Límite por transacción	50000000	<code>wallet.recargar()</code> lo valida antes de modificar el saldo; como los pagos también pasan por <code>recargar()</code> , el límite aplica a recargas y pagos.
Fondos totales	1000000	Fondos del Banco Central, inicializados una sola vez.
Reservas	0 al inicio	Se incrementan con <code>emitir_moneda()</code> ; cualquier referencia al banco ve el mismo total.

```
# Wallet consulta la configuracion global antes de modificar el saldo
def recargar(self, monto):
    BancoCentral().validar_monto(monto) # [AGREGADO] limite global
    self.__saldo += monto
    ...
```

INTEGRIDAD DE LOS DATOS

Como existe un único objeto, no hay copias de la configuración que puedan quedar desactualizadas: un cambio en el límite o en la moneda se reflejaría de inmediato en todas las billeteras. Del mismo modo, la moneda emitida desde cualquier punto del sistema se registra en las mismas reservas.

USO DE MEMORIA

Cien invocaciones de `BancoCentral()` producen un solo objeto en memoria (sección 09). Sin el patrón, las mismas cien invocaciones crean cien objetos independientes, cada uno con su propia copia de los datos.

Riesgos de no aplicar el patrón Singleton

En un sistema bancario, el gestor de la base de datos y el emisor de moneda son componentes cuyo estado debe ser único. Si cada parte del sistema creara su propia instancia, se presentarían los siguientes riesgos.

Gestor de la base de datos

Cada instancia abriría su propia conexión con la base de datos. Con muchos usuarios operando al tiempo, el servidor alcanzaría su límite de conexiones y rechazaría operaciones. Además, dos instancias podrían leer el mismo saldo, modificarlo por separado y guardar resultados distintos: la segunda escritura borraría la primera y el saldo final no correspondería a las operaciones realizadas. Con una sola instancia, todas las operaciones pasan por el mismo punto de control, que puede ordenarlas y mantener una sola conexión o un grupo controlado de conexiones.

Emisor de moneda

Si existieran dos emisores, cada uno llevaría su propio registro de reservas. El dinero emitido por uno no aparecería en el otro, y la suma real de la moneda en circulación no estaría registrada en ningún lugar. Esto equivale a dinero duplicado o no controlado, una falla grave en una entidad financiera. La verificación de la sección 09 lo demuestra: con una clase común, un banco registra 1.000 y otro 500, sin ningún total compartido; con el Singleton, ambas referencias registran el mismo total de 1.500.

Consumo de memoria

Crear un objeto por cada operación ocupa memoria con copias de la misma información. En un sistema con miles de transacciones por minuto, ese costo crece con el volumen de operaciones, mientras que con el Singleton se mantiene constante.

SÍNTESIS

Sin el patrón Singleton, los componentes críticos pierden su condición de fuente única de información: aparecen saldos inconsistentes, dinero no controlado y consumo innecesario de recursos. El patrón asegura que todo el sistema trabaje sobre el mismo estado.

Comportamiento de la implementación

El script `comprobacion_singleton.py` verifica el patrón y lo compara con una clase común equivalente. Es un recurso de apoyo para este informe y no forma parte del modelo.

```
$ python3 comprobacion_singleton.py
1. INSTANCIA UNICA
Creando el Banco Central por única vez...
  a is b: True | id(a) == id(b): True
  Objetos distintos tras 100 intentos: 1
2. ATRIBUTO ESTATICO PRIVADO
  'instancia' pertenece a la clase: True
  Acceso directo a BancoCentral.__instancia -> AttributeError (privado)
3. DATOS GLOBALES COMPARTIDOS
  Reservas vistas desde a: 1500 | desde b: 1500
  Moneda de dos billeteras distintas: COP y COP
  Recarga de 60.000.000 -> ValueError: el monto supera el limite por transaccion
4. SIN SINGLETON (comparacion)
  x is y: False
  Reservas en x: 1000 | en y: 500 -> datos inconsistentes
  Objetos distintos tras 100 intentos: 100
```

Ejecución de banco_central.py

```
$ python3 banco_central.py
Creando el Banco Central por única vez...
Es el mismo banco? True
id(banco_a): 139831256677904
id(banco_b): 139831256677904
Son el mismo objeto?: True
id(ultimo): 139831256677904
id(ultimo): 139831256677904
... (98 líneas más con el mismo identificador)
El #100 es el mismo?: True
Reservas totales (compartidas): 1500
Moneda del sistema: COP
Limite por transaccion: 50000000
```

INTERPRETACIÓN

El mensaje de creación aparece una sola vez, las cien invocaciones producen el mismo identificador de memoria y las reservas emitidas desde dos referencias distintas suman 1.500 en el mismo objeto. La clase común, en cambio, genera cien objetos y registros de reservas separados. El límite por transacción rechaza una recarga de 60.000.000, lo que confirma que la configuración global se aplica en las billeteras.

Conclusiones y oportunidades de mejora

El patrón Singleton asegura una fuente única de información. Todas las invocaciones de `BancoCentral()` trabajan sobre el mismo objeto, lo que mantiene la consistencia de los fondos, las reservas y la configuración.

La implementación depende del método `__new__`. Controlar la creación del objeto, y no su inicialización, es lo que garantiza la instancia única y evita que los datos se reinicien.

La integración con la Wallet da sentido al patrón dentro del sistema. La configuración global se aplica de forma uniforme en todas las billeteras mediante una dependencia de bajo acoplamiento.

El patrón debe usarse con criterio. Es adecuado para componentes únicos por naturaleza, como un emisor de moneda o un gestor de conexiones; su uso generalizado introduce estado global y dificulta las pruebas.

Oportunidades de mejora

MEJORA	DESCRIPCIÓN
Creación segura con hilos	Proteger la creación de la instancia con <code>threading.Lock</code> para entornos concurrentes.
Encapsular fondos y reservas	Declarar <code>fondos_totales</code> y <code>reservas</code> como privados y exponer métodos de consulta, siguiendo el mismo criterio aplicado a la configuración.
Configuración persistente	Cargar la moneda y el límite desde un archivo o una base de datos, en lugar de definirlos en el código.

REPOSITORIO

El diagrama (fuente `.drawio` y exportaciones PNG y SVG), el código fuente, el script de verificación, el README y este informe se encuentran en github.com/DavisZulu/Semana_6_Act_2.

Referencias

- Dieterichs, H. (2026). *Draw.io Integration* (versión 1.11.0) [Extensión para Visual Studio Code]. Visual Studio Marketplace. <https://marketplace.visualstudio.com/items?itemName=hediet.vscode-drawio>
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Peláez Loaiza, S. (2026). *FUNDAMENTOS-DE-SOFTWARE* [Repositorio de código]. GitHub. <https://github.com/SimonP8/FUNDAMENTOS-DE-SOFTWARE>
- Python Software Foundation. (2026). *Data model: object.__new__*. The Python Language Reference. https://docs.python.org/3/reference/datamodel.html#object.__new__
- Refactoring Guru. (2026). *Singleton*. <https://refactoring.guru/design-patterns/singleton>
- Refactoring Guru. (2026). *Singleton en Python*. <https://refactoring.guru/es/design-patterns/singleton/python/example>